Contents

Implementation Plan: Generic Raw-Data → Mesh Projection Feature (v2)

Date: 2026-05-09 **Status:** Draft v2 — supersedes v1 **Predecessor:** `data_projection_feature_plan_v1.md`
Scope: Generic, one-time, **interpolation-only** projection of structured field data onto an MFEM mesh, consumed as initial conditions / material properties by the dynamic-rupture and quasi-dynamic stack.

First concrete user: SCEC CVM-H velocity model (`safs/project_7.0_alternative/data_velocity/velocity_raw_` — 29 ASCII slices, 143 lon × 71 lat × 29 z, fields `Vp / Vs / ρ`) onto the `safs_fault_box_nwcut_{500,1000,2000}m.msh` doubled-domain meshes.

Changes from v1

#	v1 Behaviour	v2 Resolution
C-1	OOB policy could be `Clamp` / `Abort` / `ConstantFill`; default `Clamp`.	**Strict interpolation only.** `OOBPolicy` enum collapses to `{Abort}`. `Clamp` / `ConstantFill` removed from v2 (the user will resize the mesh to fit the data extent before any production run).
C-2	No upfront mesh-vs-data bbox check; OOB hits surfaced only at evaluation.	**Startup containment guard.** `DataField3D::ContainsBBox(mesh_bbox)` is called by `FieldProjector::Project` before any `mfem::Coefficient::Eval`; aborts with a printed bbox table if `mesh_bbox ⊄ data_bbox`.
C-3	Vs floor was a silent clamp at projection time.	**No silent clamping.** A configurable Vs/Vp **lower-bound guard** (`vs_min`, `vp_min`) is enforced at HDF5 load time and again at projection time; either a sub-floor cell or a sub-floor projected DOF causes a hard abort and prints the offending coordinate. The user is told to regenerate the input dataset (their stated remedy: drop sub-seafloor cells before sidecar build).
C-4	Phase 2 architecture diagram listed a `.ts` GoCAD reader as a data source.	**Removed.** `.ts` files in this project are fault surfaces (used by `nw_cut_strip.py`, never as scalar/tensor data). The Phase 2 reader registry contains ASCII-CSV (current CVM-H) plus a generic CSV-grid reader for future stress maps.
C-5	Tests listed loosely per phase.	**Test Catalog appendix.** Every public function added by this plan has a named unit test in the catalog with the input fixture, expected output, and the guard it exercises. The implementer is expected to write all of them; the reviewer checks the catalog, not just the count.
C-6	"One-time load" was implicit (assumed by reuse of `ParGridFunction`).	**Explicit one-time-load contract.** `MaterialField` is constructed once during driver startup and held by `WaveOperator` as a `const` member. A unit test (`test_field_projector_called_once`) instruments `FieldProjector::Project` with a call counter and asserts it is incremented exactly once across a `tpv102_driver` run with `--velocity-sidecar`.

Overview

Today every miniapp driver hard-codes a homogeneous (λ, μ, ρ) triple at the `WaveOperator` constructor and a closed-form (σ_n, τ_ini) field at `InitializeStateTotal`. To run real-data benchmarks (CVM-H velocity, regional stress maps from CFM, lab pore-pressure profiles) we need a **generic data projection feature** that:

1. Reads a raw external dataset in its native coordinate system and writes a normalized HDF5 sidecar in canonical CRS = UTM 11 N.
2. At driver startup, **once**, loads the sidecar, builds a 3-D structured trilinear interpolant, **verifies the mesh bbox is fully contained**, and projects onto every required mesh entity (bulk DOFs, fault QPs).
3. **Aborts loudly** on any condition that would require extrapolation, silent clamping, or quietly-physical-but-wrong material values.
4. Hands the resulting `(Par)GridFunctions` to a thin **adapter** that injects per-DOF values into the wave operator (λ(x), μ(x), ρ(x)) and the fault flux (per-DOF impedances Z_p, Z_s and pre-stress σ_n0, τ1_0, τ2_0).

The same pipeline serves Vp / Vs / ρ, σ_0_components, pore pressure, and any future scalar/tensor field; the only thing that changes per-field is the schema (named columns), the consumer adapter, and the per-field sanity bounds.

Constraints

Interface constraints (cannot change)

- **`WaveOperator(MeshType&, int order, real_t λ, real_t μ, real_t ρ, const BoundaryConfig&)`** — the existing scalar-material constructor. Extension is by overload, never by mutation. Existing TPV102 / TPV104 / TPV205 / BP5 drivers must compile and produce byte-identical results.
- **`DOFData` layout in `dynamic/fault_face_flux.hpp`** — must remain binary-compatible. Per-DOF impedances and pre-stress fields already exist; the projection adapter only writes into them.
- **`InitializeStateTotal(Vector& Q, int ndof_total, real_t σn, real_t τini)`** — keep the constant-field overload as-is; add a new field-valued overload (deferred to v3 plan).

Convention constraints

- **Canonical CRS:** UTM 11 N, EPSG:32611. Every mesh in this project is in UTM 11 N.
- **Z convention:** z = 0 at free surface, z < 0 at depth. Raw datasets using "depth, positive down" are flipped at preprocessing (`z_canon = −depth_m`).
- **File naming:** offline outputs go to `safs/<project>/data_projected/<field-set>_<mesh-tag>.h5`, mesh-coupled by construction.
- **No-touch zones (CLAUDE.md C2):** all consumer hooks in `dynamic/` must be additive; no edits to `bp5/`, `bp1/`, `bp2/`, `domain/`, `fault/`, `solver/`, or `friction/dieterich_ruina.hpp`.

Numerical / safety constraints

- **C-1 / C-2: strict interpolation only.** No extrapolation under any policy. The runtime guard is:

  ```
  data_bbox.contains(mesh_bbox)  ⇒  proceed
  data_bbox.contains(mesh_bbox)  ⇏  abort with printed bbox table
  ```

  If the user wants a larger mesh than the available data, they must either (a) regenerate the data over a wider region or (b) shrink the mesh padding. The plan does not provide a "make the bad case quiet" knob.

- **C-3: per-field sanity bounds.** Each scalar field has a configurable `[min_value, max_value]` enforced at HDF5 load and again post-projection. Defaults for the velocity field set:

Field	min	max	reason
`Vp`	2000 m/s	9000 m/s	competent rock floor (typical sediment lower bound)
`Vs`	1500 m/s	5000 m/s	competent rock floor (CVM-H sub-rock minima ≈ 120 m/s violate this; the user regenerates without seafloor)
`density`	2000 kg/m^3	3500 kg/m^3	typical crustal range

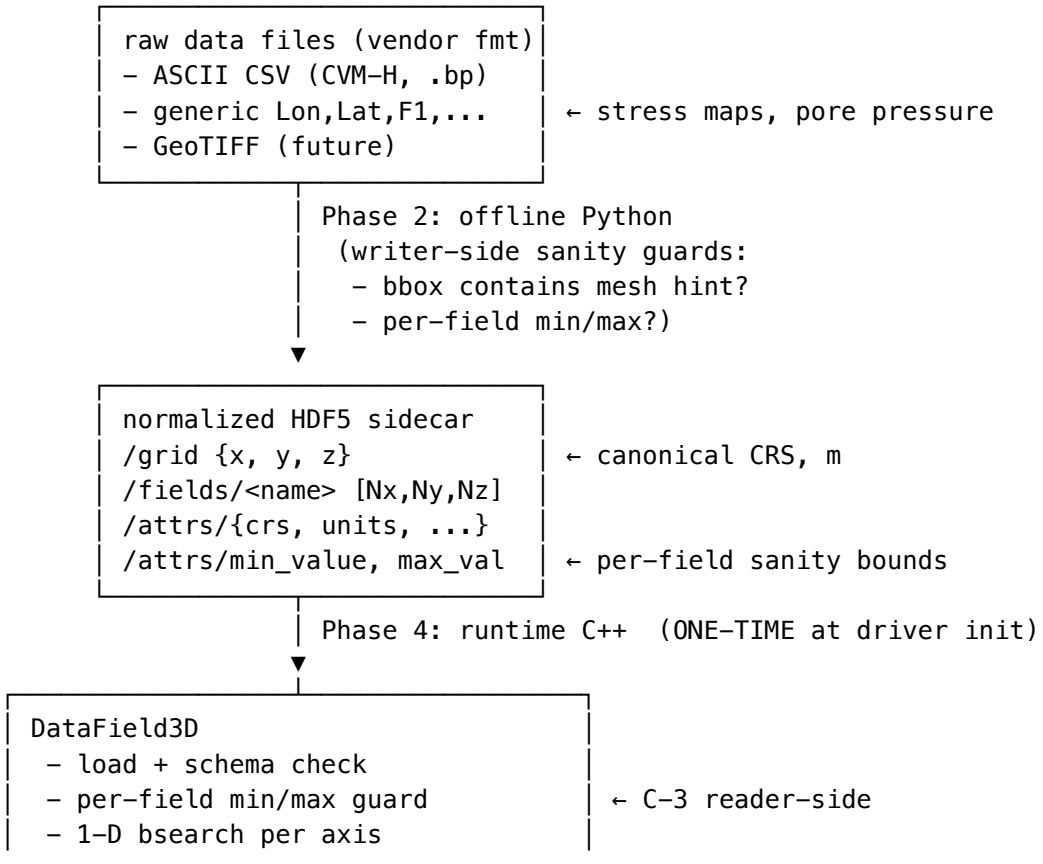
Every threshold is overridable per field via the offline `--vs-min-mps` / `--vs-max-mps` (etc.) flags AND via runtime CLI on the consumer driver. Both writer and reader perform the check independently (defense-in-depth against a stale sidecar).

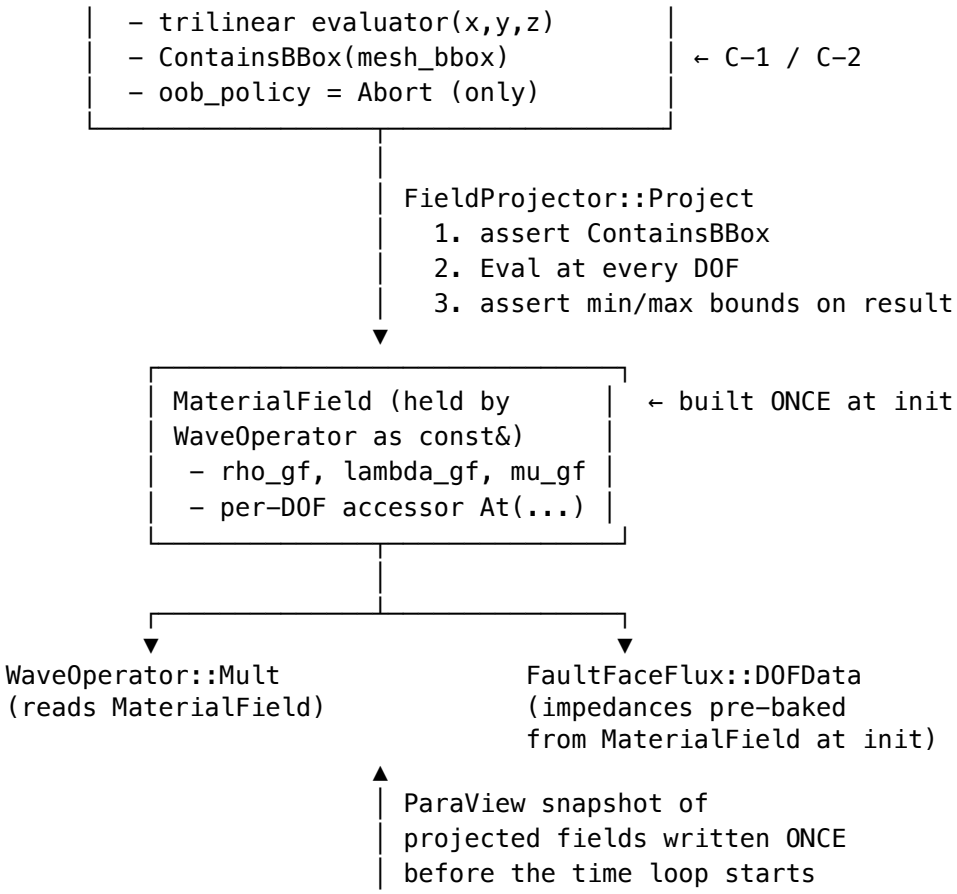
- **C-6: one-time load.** The HDF5 read, the trilinear pre-flight, and the projection happen exactly once per run, during driver init. The per-time-step hot path (`WaveOperator::Mult`, ADER recursion, friction solve) reads only the cached `ParGridFunctions`. A unit test pins this: any change that would call `FieldProjector::Project` from inside the time loop fails the test.

Performance constraints

- Total projection budget: 10 s wall-clock at 500 m mesh, np = 1.
- Per-DOF cost: O(log N_grid) trilinear evaluation. No KD-tree.
- MPI: every rank loads the same sidecar; only rank-local DOFs are projected. No collective inside the projection apart from the one-shot rank-0 summary print.

Architecture (revised)





Phase 1 — Schema & Coordinate Convention

Goal

Pin the canonical coordinate system, the file format of the normalized sidecar, and the per-field sanity-bound metadata. After this phase nothing executes — the contract for Phases 2–5 is fixed.

Files to Create

- `miniapps/seas/document/features_dev/data_projection_schema_v1.md` — the schema reference (HDF5 layout, attribute names, dtype, CRS identifiers, per-field sanity bound layout). Treat as part of the codebase: any breaking change bumps the schema version (v1, v2, …) and the runtime loader rejects mismatched versions.

Detailed Requirements

1. **Canonical CRS identifier.** Root attribute `crs = "EPSG:32611"` only. (The reserved `"local-tangent"` value for future small-domain benchmarks is not implemented in v2.)
2. **Canonical units.** `units = "m"` for x, y, z. Field-specific unit attributes per dataset.
3. **Canonical z direction.** `z_positive = "elevation"`.
4. **Sidecar structure (HDF5).**

```
<sidecar>.h5
├── attrs:
```

```
│     schema_version  = "data_projection_v1"
│     crs             = "EPSG:32611"
│     units           = "m"
│     z_positive      = "elevation"
│     created_at      = ISO-8601 timestamp
│     source          = "<comma-separated source filenames>"
│     source_crs      = "EPSG:4326"
│     mesh_tag        = "safs_fault_box_nwcut_500m"   # informational
├── /grid
│     ├── x  [Nx]  float64, strictly monotone increasing
│     ├── y  [Ny]  float64
│     └── z  [Nz]  float64
└── /fields
      ├── Vp        [Nx, Ny, Nz]  float64
      │     attrs:  units      = "m/s"
      │             min_value  = 2000.0
      │             max_value  = 9000.0
      ├── Vs        [Nx, Ny, Nz]  float64
      │     attrs:  units      = "m/s"
      │             min_value  = 1500.0
      │             max_value  = 5000.0
      ├── density   [Nx, Ny, Nz]  float64
      │     attrs:  units      = "kg/m^3"
      │             min_value  = 2000.0
      │             max_value  = 3500.0
      └── ...
```

The grid is **structured rectilinear** (separable axes). NaN cells are forbidden in v2 (writer-side guard, see Phase 2 §6).

5. **Index order.** All 3-D arrays use `[Nx, Ny, Nz]` row-major (C order, HDF5 default). Loader checks `axes-monotone` and shape match.
6. **Default OOB policy.** Sidecar carries no OOB policy attribute in v2 — the runtime is hard-wired to `OOBPolicy::Abort`. A future v2.x that re-introduces a clamp policy will bump the schema version to `data_projection_v2`.
7. **Field-set schema for the three immediate users.**

Field set	Field names	Units
`velocity`	`Vp`, `Vs`, `density`	m/s, m/s, kg/m^3
`stress`	`sigma_xx`, `sigma_yy`, `sigma_zz`, `sigma_xy`, `sigma_yz`, `sigma_xz`	Pa each
`pore_pressure`	`p`	Pa

Acceptance Criteria

- ☐ Schema doc enumerates every attribute, dtype, dimension order.
- ☐ Schema doc explicitly states “NaN forbidden”, “OOB policy = abort only”, “per-field min_value / max_value mandatory”.
- ☐ Phase 2 (writer) and Phase 3 (reader) requirements both cite the schema doc.

Dependencies

- Depends on: nothing.
- Required by: Phases 2, 3, 4, 5.

Phase 2 — Offline Preprocessor (Python)

Goal

Convert the CVM-H ASCII slices in `safs/project_7.0_alternative/data_velocity/velocity_raw_*.bp` to a single schema-conforming HDF5 sidecar, with **writer-side guards** for per-field sanity bounds and an **optional mesh-bbox containment hint**.

Files to Create

- `safs/project_7.0_alternative/code_preprocess/data_projection/__init__.py`
- `safs/project_7.0_alternative/code_preprocess/data_projection/raw_readers.py` — readers:
  - `read_cvmh_ascii(paths: list[Path]) -> RawSliceStack`
  - `read_csv_grid(path: Path, columns: list[str], crs: str) -> RawGrid`
- `safs/project_7.0_alternative/code_preprocess/data_projection/crs.py` — `geographic_to_utm11n(lon, lat) -> (x, y)`
- `safs/project_7.0_alternative/code_preprocess/data_projection/sidecar.py` — `write_sidecar(out_path, x, y, z, fields, attrs, field_bounds) -> None`
- `safs/project_7.0_alternative/code_preprocess/data_projection/bbox_check.py` — `mesh_msh_bbox(msh_path: Path) -> dict` and `assert_grid_contains_mesh(x, y, z, mesh_bbox) -> None`
- `safs/project_7.0_alternative/code_preprocess/data_projection/build_velocity_cvmh.py` — top-level driver: ASCII → UTM rectilinear → HDF5 sidecar.
- `safs/project_7.0_alternative/code_preprocess/data_projection/test_data_projection.py` — pytest suite.

Files to Modify

- None — all new code under `code_preprocess/data_projection/`.

Detailed Requirements

1. **CVM-H reader (`raw_readers.read_cvmh_ascii`).**

```python
def read_cvmh_ascii(paths: list[Path]) -> RawSliceStack:
    """Return a RawSliceStack with attributes:
      lon_grid : (N_lon,) float64, strictly increasing
      lat_grid : (N_lat,) float64, strictly increasing
      depths_m : (N_z,)   float64, monotone increasing (positive=down)
      fields   : dict[name -> (N_lon, N_lat, N_z) float64]

    Each path is one slice; depth comes from the `# Depth(m): N`
    header, NOT the filename (the existing data uses misleading
    `X.Ykm` filename labels equal to depth_km/10).
    """
```

   Per-file parser steps:

   - Read # header until column line `# Lon,Lat,Vp(m/s),Vs(m/s),Density(kg/m^3)`.

- Capture `Depth(m)`, `Spacing(degree)`, `Lon_pts`, `Lat_pts`, `Total_pts`, `Lat1, Lon1, Lat2, Lon2`.
- `assert Lat_pts * Lon_pts == Total_pts`.
- Read body via `numpy.loadtxt`.
- **Pin orientation.** First sample row must equal `(Lon1, Lat1, …)`, last row must equal `(Lon2, Lat2, …)`; abort otherwise.
- Reshape to `(Lon_pts, Lat_pts)` row-major (lon outer, lat inner).
- Across slices: assert all share an identical `(lon_grid, lat_grid)`. Stack along z sorted by `Depth(m)` ascending.

2. **CRS conversion (`crs.geographic_to_utm11n`).**

```
def geographic_to_utm11n(lon: np.ndarray, lat: np.ndarray
                         ) -> tuple[np.ndarray, np.ndarray]:
    """Vectorised lon/lat → UTM 11 N. Uses pyproj.Transformer with
    always_xy=True. Raises if pyproj missing."""
```

3. **Resampling strategy (`build_velocity_cvmh.py`).**

   1. Read slices.
   2. Convert (lon, lat) → (x_utm, y_utm); irregular 2-D point cloud per slice.
   3. Build a rectilinear UTM grid covering the convex hull of the sample points, with $\Delta x = \Delta y =$ `grid_spacing_m` (default `1500.0`).
   4. Per slice, `scipy.interpolate.LinearNDInterpolator` from `(x_utm, y_utm)` to the rectilinear grid; **NaN cells outside the convex hull will trigger a writer-side guard, see §6**.
   5. Convert depth → elevation: `z_canon[k] = −depths_m[k]`. Sort ascending.
   6. Apply per-field sanity guard (see §6) before write.
   7. Write via `sidecar.write_sidecar`.

4. **Sidecar writer (`sidecar.write_sidecar`).**

```
def write_sidecar(out_path: Path,
                  x: np.ndarray, y: np.ndarray, z: np.ndarray,
                  fields: dict[str, np.ndarray],
                  attrs: dict[str, Any],
                  field_bounds: dict[str, tuple[float, float]]) -> None:
    """Write the schema-v1 HDF5. Asserts:
      - x, y, z strictly monotone increasing
      - every fields[name].shape == (len(x), len(y), len(z))
      - attrs contains required schema-v1 keys
      - field_bounds covers every key in `fields`
      - NO NaN in any field array
      - every value in [field_bounds[name][0], field_bounds[name][1]]
    """
```

5. **Mesh-bbox containment check (`bbox_check.assert_grid_contains_mesh`).**

```
def assert_grid_contains_mesh(x: np.ndarray, y: np.ndarray,
                              z: np.ndarray, mesh_bbox: dict) -> None:
    """Abort with a printed bbox table if the rectilinear grid
    (x, y, z) does NOT strictly contain the mesh bbox.
    mesh_bbox is a dict with keys
    {xmin, xmax, ymin, ymax, zmin, zmax} in canonical CRS.
    """
```

`mesh_msh_bbox(msh_path)` shells out to `meshio` (already in the pythonenv environment per `feedback_sbatch_modules.md`):

```
import meshio, numpy as np
m = meshio.read(str(msh_path))
P = np.asarray(m.points)
return dict(xmin=float(P[:,0].min()), xmax=float(P[:,0].max()),
            ymin=float(P[:,1].min()), ymax=float(P[:,1].max()),
            zmin=float(P[:,2].min()), zmax=float(P[:,2].max()))
```

6. **Writer-side sanity guards (CRITICAL — Phase 2 contract).** `build_velocity_cvmh.py` performs these checks BEFORE calling `write_sidecar`, in this order:

   G-1: NaN guard. No NaN allowed in any field after resample. This catches the "mesh extends beyond the data hull" failure mode directly in the offline step. If any cell is NaN, abort with:

   ```
   ERROR: <Nfield> NaN cells in field 'Vp' after resample.
          The rectilinear UTM grid extends beyond the source data hull.
          Either:
            (a) shrink the requested grid bounds via --xmin/--xmax/
                --ymin/--ymax, OR
            (b) extend the source raster to cover the requested region.
          Source hull bbox (UTM 11 N): x=[..., ...], y=[..., ...]
          Requested grid bbox       : x=[..., ...], y=[..., ...]
   ```

   G-2: Per-field sanity bounds. For each field, fail loudly if any cell is below `min_value` or above `max_value`:

   ```
   ERROR: 124 cells of field 'Vs' below min_value=1500 m/s
          (min observed = 120.69 m/s at (lon, lat, depth) = (...)).
          The CVM-H sub-seafloor mask was likely not stripped from the
          input. Regenerate the source ASCII slices excluding marine /
          water-saturated cells, or raise --vs-min-mps explicitly.
   ```

   G-3: Optional mesh-bbox hint. If the user passes `--mesh-msh PATH`, call `assert_grid_contains_mesh`. This is the writer-side counterpart to the runtime guard in Phase 4 — catches the bad case as early as possible, before the runtime gate.

7. **CLI (`build_velocity_cvmh.py`).**

   ```
   python -m data_projection.build_velocity_cvmh \
       --raw-dir       data_velocity \
       --out-path      data_projected/velocity_safs.h5 \
       --grid-dx       1500 \
       --vp-min-mps    2000 \
       --vp-max-mps    9000 \
       --vs-min-mps    1500 \
       --vs-max-mps    5000 \
       --rho-min-kgm3  2000 \
       --rho-max-kgm3  3500 \
       --mesh-msh      ../code_meshing/safs_fault_box_nwcut_500m.msh \
       --verbose
   ```

   Failure of any of G-1 / G-2 / G-3 returns a non-zero exit code. No "force" / "no-check" override flag exists in v2 — by design.

Edge Cases to Handle

- **Filename vs header depth mismatch** — header is the truth.
- **Lon order reversed in some slices** — defensive sort by lon ascending after parsing each slice.
- **Per-row column count mismatch** — abort with line number.

Acceptance Criteria

- ☐ `python -m data_projection.build_velocity_cvmh --raw-dir data_velocity --out-path data_projected/velocity_safs.h5 --mesh-msh ../code_meshing/safs_fault_box_nwcut_2000r` either succeeds (if user has regenerated CVM-H without seafloor) or fails with a clear G-2 message pointing to sub-floor Vs cells (the current real input — confirms guard fires).
- ☐ `h5dump -A` on the output shows `schema_version = "data_projection_v1"`, `crs = "EPSG:32611"`, `z_positive = "elevation"`, and per-field `min_value` / `max_value`.
- ☐ No NaN in any `/fields/*` dataset (writer guard G-1).
- ☐ `pytest -q test_data_projection.py` is green (synthetic fixtures + the smoke test). Test catalog in §Test Catalog.
- ☐ No edit to existing seas / preprocess files outside `code_preprocess/data_projection/`.

Dependencies

- Depends on: Phase 1.
- Required by: Phase 4 (loader needs a real sidecar to test).

Phase 3 — Runtime Loader & Interpolant (`DataField3D`)

Goal

A header-only C++ class that loads a schema-v1 sidecar **once**, performs schema and per-field sanity checks at load time, and evaluates the trilinear interpolant of any of its `/fields/<name>` at arbitrary `(x, y, z)` in UTM 11 N. Any out-of-bbox query aborts.

Files to Create

- `miniapps/seas/io/data_field_3d.hpp`
- `miniapps/seas/io/data_field_3d.cpp` (only if HDF5 implementation forces non-template code out of the header)
- `miniapps/seas/test/test_data_field_3d.cpp`

Files to Modify

- `miniapps/seas/CMakeLists.txt`, `miniapps/seas/Makefile` — register new sources, link `${HDF5_C_LIBRARIES}` (already a transitive dep via MFEM but cite explicitly).

Detailed Requirements

1. **Class definition.**

```cpp
namespace mfem::seas {

/// v2: only Abort is supported. Clamp/ConstantFill removed.
enum class OOBPolicy : int { Abort };

class DataField3D
{
public:
    /// Load a single named field from a schema-v1 HDF5 sidecar.
    /// Verifies at load time:
    ///   - schema_version == "data_projection_v1"
    ///   - crs == "EPSG:32611"
```

```
    ///   - z_positive == "elevation"
    ///   - axes monotone increasing
    ///   - dataset shape == (Nx, Ny, Nz)
    ///   - NO NaN in dataset
    ///   - every cell within [min_value, max_value]
    /// Any failure -> MFEM_ABORT with precise message.
    DataField3D(const std::string& sidecar_path,
                const std::string& field_name,
                OOBPolicy oob = OOBPolicy::Abort);

    /// Evaluate the trilinear interpolant. Aborts on out-of-bbox.
    real_t Evaluate(real_t x, real_t y, real_t z) const;

    /// Inclusive bbox in canonical CRS:
    /// {xmin, xmax, ymin, ymax, zmin, zmax}
    const std::array<real_t, 6>& BBox() const { return bbox_; }

    /// Mesh-bbox containment check. Returns true iff the supplied
    /// box is FULLY contained (with optional epsilon). Used by
    /// FieldProjector::Project as a startup gate.
    bool ContainsBBox(real_t xmin, real_t xmax,
                      real_t ymin, real_t ymax,
                      real_t zmin, real_t zmax,
                      real_t eps = 0.0) const;

    const std::string& FieldName() const { return field_name_; }
    const std::string& Units()     const { return units_; }
    real_t MinValue()              const { return min_value_; }
    real_t MaxValue()              const { return max_value_; }

private:
    std::vector<real_t> x_, y_, z_;          // monotone axes
    mfem::Array3D<real_t> data_;              // (Nx, Ny, Nz)
    std::array<real_t, 6> bbox_;
    std::string field_name_, units_;
    real_t min_value_, max_value_;
    OOBPolicy oob_policy_;

    int find_index_(const std::vector<real_t>& axis, real_t v) const;
};
} // namespace mfem::seas
```

2. **Schema check at load.**

```
MFEM_VERIFY(schema_version == "data_projection_v1",
            "DataField3D: unexpected schema_version='" <<
            schema_version << "', expected 'data_projection_v1'");
MFEM_VERIFY(crs == "EPSG:32611",
            "DataField3D: only EPSG:32611 supported, got '" <<
            crs << "'");
MFEM_VERIFY(z_positive == "elevation",
            "DataField3D: only z_positive='elevation' supported");
```

3. **Per-field sanity check at load (C-3 reader-side).**

```
for (int idx = 0; idx < data_.NumElements(); ++idx) {
    real_t v = data_[idx];
    MFEM_VERIFY(!std::isnan(v),
                "DataField3D: NaN at flat index " << idx <<
                " in field '" << field_name_ << "'. Sidecar must " <<
                "be regenerated; v1 schema forbids NaN.");
    MFEM_VERIFY(v >= min_value_ && v <= max_value_,
                "DataField3D: value " << v << " out of declared " <<
                "range [" << min_value_ << ", " << max_value_ <<
                "] in field '" << field_name_ << "'. Regenerate " <<
                "the source dataset.");
}
```

4. **`ContainsBBox` (C-2 / C-1 enabler).**

```
bool DataField3D::ContainsBBox(real_t xmin, real_t xmax,
                               real_t ymin, real_t ymax,
                               real_t zmin, real_t zmax,
                               real_t eps) const
{
    return (xmin >= bbox_[0] - eps) && (xmax <= bbox_[1] + eps)
        && (ymin >= bbox_[2] - eps) && (ymax <= bbox_[3] + eps)
        && (zmin >= bbox_[4] - eps) && (zmax <= bbox_[5] + eps);
}
```

5. **Trilinear evaluation.** Standard formula:

```
i = bsearch(x_, x); j = bsearch(y_, y); k = bsearch(z_, z);
u = (x − x_[i]) / (x_[i+1] − x_[i]);    // ∈ [0, 1]
v = (y − y_[j]) / (y_[j+1] − y_[j]);
w = (z − z_[k]) / (z_[k+1] − z_[k]);
f = (1−u)(1−v)(1−w) f_000  +  u(1−v)(1−w) f_100
  + (1−u) v   (1−w) f_010  +  u  v   (1−w) f_110
  + (1−u)(1−v) w    f_001  +  u(1−v) w    f_101
  + (1−u) v   w     f_011  +  u  v   w    f_111
```

   Out-of-bbox at any axis → abort with the offending coordinate.

6. **Binary search.** `find_index_` uses `std::upper_bound` on `axis.begin()..axis.end()−1`, returning an index `i` with `x_[i] <= v <= x_[i+1]` for `v ∈ [x_[0], x_[Nx−1]]`. Out-of-range triggers abort:

```
MFEM_ABORT("DataField3D::Evaluate: query (" << x << ", " << y <<
           ", " << z << ") is OUTSIDE field '" << field_name_ <<
           "' bbox = [" << bbox_[0] << ", " << bbox_[1] << "] x [" <<
           bbox_[2] << ", " << bbox_[3] << "] x [" << bbox_[4] <<
           ", " << bbox_[5] << "]. v2 schema enforces interpolation−" <<
           "only; either shrink the mesh or regenerate the sidecar.");
```

7. **HDF5.** Use the C API (`H5Fopen`, `H5Dread`, `H5Aread`); copy the include pattern from MFEM's own HDF5 use.

8. **Thread safety.** All reads are const; counters removed (Abort policy makes them moot).

Edge Cases to Handle

- **Sidecar missing**: `MFEM_ABORT("DataField3D: cannot open sidecar '<path>'")`.
- **Field name not present**: list available `/fields/*` in the abort message.

- **Single-cell axis** (Nx == 1): treat as exact match required; any off-axis query is OOB → abort.
- **Empty dataset**: abort at load.

Acceptance Criteria

- ☐ All tests in §Test Catalog T−3−∗ pass at np = 1.
- ☐ No edits to `bp5/`, `bp1/`, `bp2/`, `domain/`, `friction/`, `dynamic/`. (Phase 5 will do those edits, additively.)

Dependencies

- Depends on: Phase 1, Phase 2 (for at least one real sidecar to test against).
- Required by: Phase 4, 5.

Phase 4 — Mesh Projector (`FieldCoefficient` + `FieldProjector`)

Goal

Bind a `DataField3D` to MFEM via `Coefficient` so `ParGridFunction::ProjectCoefficient` does the bulk-DOF projection in one call. **Pre-flight the projection** with a `DataField3D::ContainsBBox` check that aborts if the mesh is not fully inside the data extent. Post-project, **assert per-field min/max bounds** on the resulting GridFunction values (defense-in-depth against numerical surprises like the trilinear stencil producing a value below `min_value` due to extreme local gradients — should never happen in real CVM-H but is cheap to verify).

Files to Create

- `miniapps/seas/io/field_coefficient.hpp` — `FieldCoefficient`, `FieldProjector` (header-only).
- `miniapps/seas/test/test_field_coefficient.cpp`
- `miniapps/seas/test/test_field_projector.cpp`

Files to Modify

- `miniapps/seas/CMakeLists.txt`, `Makefile` — register new headers and tests.

Detailed Requirements

1. **`FieldCoefficient` (scalar).**

```
class FieldCoefficient : public mfem::Coefficient
{
public:
    FieldCoefficient(const DataField3D& field,
                     real_t scale = 1.0,
                     real_t offset = 0.0)
        : field_(field), scale_(scale), offset_(offset) {}

    real_t Eval(mfem::ElementTransformation& T,
                const mfem::IntegrationPoint& ip) override
    {
        Vector x;
        T.Transform(ip, x);
        return scale_ * field_.Evaluate(x[0], x[1], x[2]) + offset_;
    }
```

```
private:
    const DataField3D& field_;
    real_t scale_, offset_;
};
```

No `floor_` parameter — silent flooring is forbidden in v2 (C-3). Composing $\mu = \rho$ V_s^2 etc. happens via `mfem::ProductCoefficient`.

2. **`FieldProjector::Project` (scalar field).**

```
struct FieldProjectionResult
{
    std::shared_ptr<mfem::ParGridFunction> gf;
    real_t min_value;       // observed in projected GF
    real_t max_value;
};

class FieldProjector
{
public:
    /// Pre-flights mesh containment, projects, post-checks min/max.
    /// Aborts loudly on any guard violation.
    static FieldProjectionResult Project(
        const DataField3D&           field,
        mfem::ParFiniteElementSpace& target_fes,
        real_t scale = 1.0,
        real_t offset = 0.0);

    /// Convenience for the velocity field set: returns three GFs
    /// in the order {ρ, λ, μ} given Vp, Vs, ρ in the sidecar.
    /// λ and μ are derived: μ = ρ V_s^2, λ = ρ V_p^2 - 2 μ.
    /// Per-field sanity bounds are inherited from the sidecar
    /// AND re-checked on the derived λ, μ (needs explicit lo/hi).
    struct VelocityFields {
        std::shared_ptr<mfem::ParGridFunction> rho, lambda, mu;
        real_t min_vp, max_vp, min_vs, max_vs;
        real_t min_lambda, max_lambda, min_mu, max_mu;
    };
    static VelocityFields ProjectVelocity(
        const std::string&           sidecar_path,
        mfem::ParFiniteElementSpace& target_fes,
        real_t lambda_min_pa = 1.0e9,
        real_t lambda_max_pa = 1.0e11,
        real_t mu_min_pa     = 1.0e9,
        real_t mu_max_pa     = 1.0e11);

    /// Test-only counter; incremented by Project() / ProjectVelocity()
    /// each call. test_field_projector_called_once asserts this is
    /// exactly 1 across a tpv102_driver run with --velocity-sidecar.
    static int CallCount() { return call_count_; }
    static void ResetCallCount() { call_count_ = 0; }

private:
    static std::atomic<int> call_count_;
};
```

3. **Pre-flight bbox guard.** First lines of `Project`:

```
const auto& mesh = *target_fes.GetParMesh();
real_t xmin, xmax, ymin, ymax, zmin, zmax;
ComputeMeshBBox(mesh, xmin, xmax, ymin, ymax, zmin, zmax);  // helper

if (!field.ContainsBBox(xmin, xmax, ymin, ymax, zmin, zmax))
{
    const auto& fb = field.BBox();
    MFEM_ABORT(
        "FieldProjector::Project: mesh bbox is NOT contained in " <<
        "field '" << field.FieldName() << "' data bbox.\n" <<
        "  mesh bbox  (UTM 11 N, m): x=[" << xmin << ", " << xmax <<
        "] y=[" << ymin << ", " << ymax << "] z=[" << zmin << ", " <<
        zmax << "]\n" <<
        "  data bbox  (UTM 11 N, m): x=[" << fb[0] << ", " << fb[1] <<
        "] y=[" << fb[2] << ", " << fb[3] << "] z=[" << fb[4] << ", " <<
        fb[5] << "]\n" <<
        "v2 schema enforces interpolation-only; either shrink the " <<
        "mesh padding or regenerate the sidecar over a wider region.");
}
```

`ComputeMeshBBox` is a private helper that takes an `MPI_Allreduce` on `MIN`/`MAX` to handle `ParMesh`.

4. **Post-projection sanity bounds.** After `gf->ProjectCoefficient(coef)`:

```
real_t lo, hi;
ComputeGridFunctionMinMax(*gf, lo, hi);   // Allreduce MIN / MAX
MFEM_VERIFY(lo >= field.MinValue() && hi <= field.MaxValue(),
            "FieldProjector::Project: projected '" << field.FieldName() <<
            "' violates per-field bounds. observed=[" << lo << ", " <<
            hi << "], declared=[" << field.MinValue() << ", " <<
            field.MaxValue() << "]. Either widen the bounds in the " <<
            "sidecar or regenerate the source data.");
```

5. **`ProjectVelocity` adds derived-quantity guards.** After computing μ`_gf` and λ`_gf` via `ProductCoefficient` chains:
   - assert `min(`μ`_gf) >= mu_min_pa`, `max(`μ`_gf) <= mu_max_pa`
   - assert `min(`λ`_gf) >= lambda_min_pa`, `max(`λ`_gf) <= lambda_max_pa` Defaults `mu_min = 1` GPa (∼ very loose sediment lower bound at Vs_min = 1500, ρ_min = 2000 → 4.5 GPa, so 1 GPa is conservative); `mu_max = 100` GPa (∼ Vs = 5000, ρ = 3500 → 87.5 GPa).
6. **One-time-load enforcement (C-6).** `call_count_` is a static atomic; `Project` and `ProjectVelocity` increment it. Test `test_field_projector_called_once` (Test Catalog T-4-7) instruments a tpv102 driver run and asserts exactly one increment.
7. **ParaView snapshot.** After `Project` / `ProjectVelocity`, the driver writes a ParaView VTU pair for visual QA before the simulation starts. Driver-side wiring (Phase 5) — the projector itself stays pure.

Edge Cases to Handle

- **Mesh entirely outside data**: pre-flight aborts with the bbox table. (Real-world: if the user forgets `--double-domain` and the sidecar was built for the doubled mesh, this fires.)
- **`ParFiniteElementSpace` order = 0**: assert `target_fes.GetOrder(0) >= 1`.
- **Heterogeneous element types**: the projection works elementwise via `IntegrationPoint::Transform`; no extra handling needed.

Acceptance Criteria

- ☐ All tests in §Test Catalog T−4−∗ pass at np = 1 and np = 4.
- ☐ On the 2000 m mesh with a synthetic schema-v1 sidecar (constructed in test fixture to contain the mesh bbox), `ProjectVelocity` returns three GFs with all values within the declared bounds.
- ☐ Pre-flight abort fires when sidecar is too small (synthetic test with a sidecar bbox slightly smaller than the mesh; abort message contains both bbox tables).

Dependencies

- Depends on: Phase 3.
- Required by: Phase 5.

Phase 5 — Consumer Adapters (`WaveOperator`, fault DOFs)

Goal

Wire the projected $(\rho,\ \lambda,\ \mu)$ GFs into the dynamic-rupture stack with a **hard one-time-load contract**. After this phase a `tpv102_driver` invoked with `--velocity-sidecar PATH` initialises with a heterogeneous medium, the simulation runs to completion at np ≥ 1, and the time loop performs **zero** sidecar reads or projection calls.

Files to Create

- `miniapps/seas/dynamic/heterogeneous_material.hpp` — `MaterialField` struct holding the three `ParGridFunctions`.
- `miniapps/seas/dynamic/heterogeneous_material.cpp` — derived quantities (impedances, per-element cp/cs).
- `miniapps/seas/test/test_heterogeneous_material.cpp`
- `miniapps/seas/test/test_tpv102_with_velocity_sidecar.cpp`

Files to Modify

- `miniapps/seas/dynamic/wave_operator.hpp` — add a second constructor taking `const MaterialField&`. The original $(\lambda,\ \mu,\ \rho)$ ctor remains and forwards to a constant `MaterialField`.
- `miniapps/seas/dynamic/wave_operator.inl` — replace scalar `lambda_, mu_, rho_` reads inside `Mult` and the ADER recursion with `material_.At(elem, dof, λ, μ, ρ)`. Constant case is byte-identical to today.
- `miniapps/seas/dynamic/fault_face_flux.cpp` — add helper `InitializeImpedancesFromMaterial(DOFData* dof, const MaterialField&, const FaultGeometry&)` that fills `Zp_plus, Zp_minus, …` once at init.
- `miniapps/seas/drivers/tpv102_driver.cpp` — add CLI flag `--velocity-sidecar PATH`. Default off → byte-identical to today.

Detailed Requirements

1. **`MaterialField` struct.**

```cpp
namespace mfem::seas {
struct MaterialField
{
    enum class Mode : int { Constant, GridFunction };
    Mode mode = Mode::Constant;
```

```
    // Scalar fallback (mode == Constant)
    real_t lambda_const = 0, mu_const = 0, rho_const = 0;

    // Heterogeneous (mode == GridFunction). Same FES across all three.
    std::shared_ptr<mfem::ParGridFunction> rho_gf, lambda_gf, mu_gf;

    /// Element-and-DOF-local accessor used inside ADER assembly.
    /// MUST be inline + branch-free on hot path.
    inline void At(int elem, int dof,
                   real_t& lambda_out, real_t& mu_out, real_t& rho_out) const;

    /// Per-element max c_p over its DOFs (for CFL).
    real_t MaxCpInElement(int elem) const;
};
} // namespace mfem::seas
```

`At` dispatches on `mode` via a `__builtin_expect`-tagged branch; the constant path is the predicted hot case for current production drivers (TPV102 / TPV104 / TPV205 / BP5).

2. **`WaveOperator` ctor changes.**

```
// New (heterogeneous):
WaveOperator(MeshType& mesh, int order,
             const MaterialField& material,
             const BoundaryConfig& bc);

// Existing (scalar) — reduces to:
WaveOperator(MeshType& mesh, int order,
             real_t lambda, real_t mu, real_t rho,
             const BoundaryConfig& bc)
    : WaveOperator(mesh, order,
                   MaterialField{Mode::Constant, lambda, mu, rho,
                                 nullptr, nullptr, nullptr},
                   bc) {}
```

`material_` is stored by value as a `const` member.

3. **ADER assembly change.** In `wave_operator.inl::Mult` and the ADER Cauchy–Kovalevskaya recursion, every read of (λ, μ, ρ) is replaced with `material_.At(elem, dof, λ, μ, ρ)`. The flux matrices remain DOF-local; existing per-element precomputation is preserved when `material_.mode == Constant` for byte-identical regression.

4. **CFL.** Replace constant `c_p` in `WaveOperator::ComputeMaxDt` with `material_.MaxCpInElement(elem)`; reduce via `MPI_Allreduce(MIN, dt_local)`.

5. **Fault impedances (`InitializeImpedancesFromMaterial`).** Walks fault QPs (handle from `FaultBasis::GetFaultQPs()`); evaluates the three GFs at each QP (face-averaged ±); writes:

```
dof.Zp_plus  = ρ⁺ · √((λ⁺ + 2μ⁺) / ρ⁺);
dof.Zp_minus = ρ⁻ · √((λ⁻ + 2μ⁻) / ρ⁻);
dof.Zs_plus  = ρ⁺ · √(μ⁺ / ρ⁺);
dof.Zs_minus = ρ⁻ · √(μ⁻ / ρ⁻);
dof.eta_p    = (Zp_plus * Zp_minus) / (Zp_plus + Zp_minus);
dof.eta_s    = (Zs_plus * Zs_minus) / (Zs_plus + Zs_minus);
```

Called once at init; never per-step. The TPV102 constant case is bit-equivalent.

6. **Driver wiring (`tpv102_driver.cpp`).**

- Parse `--velocity-sidecar PATH`.
- If unset: byte-identical to current behaviour.
- If set:
  1. `FieldProjector::ResetCallCount();`
  2. `auto vf = FieldProjector::ProjectVelocity(path, target_fes, …);`
  3. Build `MaterialField mat = MakeMaterialField(vf);`
  4. Pass `mat` to the new `WaveOperator` ctor.
  5. `InitializeImpedancesFromMaterial(dof_data, mat, fault_geom);`
  6. Write the ParaView snapshot of the projected fields: `ParaviewWriter::WriteOnce(out_dir + "/projected_fields", mat);`
  7. Inside the time loop, **assert** `FieldProjector::CallCount() == 1`. (Compiled out in Release via `MFEM_ASSERT`.)

7. **One-time-load contract test (C-6).** `test_field_projector_called_once` builds a tiny tpv102-style driver with `--velocity-sidecar`, runs 5 time steps, then asserts `FieldProjector::CallCount() == 1`.

Edge Cases to Handle

- **Mesh DOF outside CVM-H hull.** Cannot happen in v2 — pre-flight guard in Phase 4 already aborted at startup.
- **Vs floor activation.** Cannot happen in v2 — load-time and post-project sanity checks already aborted with the offending cell.
- **Fault QP straddling hull edge.** Same as above — pre-flight guard catches.

Acceptance Criteria

- ☐ `mpirun -np 1 tpv102_driver --mesh tpv102_mesh.msh` (no `--velocity-sidecar`) produces output that matches a stored reference file byte-for-byte.
- ☐ `mpirun -np 4 tpv102_driver --mesh safs_fault_box_nwcut_2000m.msh --velocity-sidecar data_projected/velocity_safs.h5` runs to completion without abort, and the rupture front in the ParaView output is visibly slowed in the high-Vp basement.
- ☐ CFL reduces correctly: Δt_CFL reported by the driver shrinks compared to the homogeneous TPV102 baseline.
- ☐ Existing test suite (`make test`) is green.
- ☐ All tests in §Test Catalog T−5−∗ pass.

Dependencies

- Depends on: Phase 4.
- Required by: any future per-DOF-stress / pore-pressure consumer (v3).

Phase 6 — Generalisation Hooks (deferred to plan v3)

Roadmap, not action item:

- **Stress field consumer.** `InitializeStateTotal_Field` overload in `dynamic/tpv102_setup_total.hpp` taking six `ParGridFunctions` for the σ tensor. Same load-time and projection-time guards.
- **Pore pressure consumer.** Effective normal stress in `friction/dieterich_ruina.hpp` via an additive hook in `dynamic/effective_stress.hpp` (no edit to `dieterich_ruina.hpp`, per CLAUDE.md C2).
- **Spatially varying friction parameters.** `a(x)`, `Dc(x)` GFs consumed by `InitializeFaultDOFs_*`.
- **Generic config plumbing.** TOML stanza

```
[data_projection]
velocity = "data_projected/velocity_safs.h5"
stress   = "data_projected/stress_cfm_safs.h5"
pore     = "data_projected/pore_baseline.h5"
```

- **Higher-order interpolation.** Tricubic / monotone Hermite for fields where C^1 continuity matters (stress).

Test Catalog (every public function gets a test)

Format: T–<phase>–<n> | <function under test> | <fixture> | <expected> | <guard exercised>.

Phase 2 — `test_data_projection.py`

ID	Function	Fixture	Expected	Guard
T-2-1	`read_cvmh_ascii`	2-slice synthetic with known Lat1/Lon1/Lat2/Lon2 corners	`lon_grid[0] == Lon1`, `lat_grid[-1] == Lat2`	orientation
T-2-2	`read_cvmh_ascii`	filename `_5km.bp` with header `Depth(m): 50000`	returned `depths_m[0] == 50000`	filename trap
T-2-3	`read_cvmh_ascii`	three slices with mismatched lon grids	abort with diff message	grid coherence
T-2-4	`geographic_to_utm` ↔ inverse	1024 random (lon, lat) pairs	round-trip back to lon/lat within 1e-7°	CRS round-trip
T-2-5	`geographic_to_utm`	known anchor (lon=−118.1677, lat=33.3428)	UTM = published value within 0.1 m	CRS calibration
T-2-6	`assert_grid_contains`	grid x∈[0,10], mesh x∈[1,9]	returns	C-2 happy path
T-2-7	`assert_grid_contains`	grid x∈[0,10], mesh x∈[-1,9]	abort with bbox table	C-2 sad path
T-2-8	`mesh_msh_bbox`	`safs_fault_box_nwcut_2000m.msh`	matches the doubled-domain bbox to ±1 m	meshio integration
T-2-9	`write_sidecar`	constant Vp = 4000 over a 4×4×4 grid	h5 round-trip preserves exact values + attrs	I/O round-trip
T-2-10	`write_sidecar`	one cell of Vp = NaN	abort	C-3 G-1 (NaN)
T-2-11	`write_sidecar`	one cell of Vs = 100 m/s with `vs_min = 1500`	abort	C-3 G-2 (low)
T-2-12	`build_velocity_cvmh` (smoke)	real `data_velocity/` (skip if absent)	either succeeds OR fails with G-2 pointing to seafloor	E2E on real data

Phase 3 — `test_data_field_3d.cpp`

ID	Function	Fixture	Expected	Guard
T-3-1	`DataField3D` ctor	sidecar with `schema_version="data_projection_v0"`	abort	schema gate

ID	Function	Fixture	Expected	Guard
T-3-2	`DataField3D` ctor	sidecar with `crs="EPSG:4326"`	abort	CRS gate
T-3-3	`DataField3D` ctor	sidecar with NaN at one cell	abort	C-3 reader-side
T-3-4	`DataField3D` ctor	sidecar with Vp = 100 m/s, declared min = 2000	abort with offending coordinate	C-3 reader-side
T-3-5	`DataField3D::Evaluate`	linear field f(x,y,z) = ax+by+cz+d on a 3×3×3 grid	exact match at 1024 random in-bbox points within 1e-12	trilinear exactness
T-3-6	`DataField3D::Evaluate`	(x, y, z) = exact grid corner	returns the grid sample	corner case
T-3-7	`DataField3D::Evaluate`	(x, y, z) = midpoint between two samples	returns arithmetic mean	midpoint
T-3-8	`DataField3D::Evaluate`	OOB query (x = bbox xmax + 1)	abort	C-1
T-3-9	`DataField3D::Contains`	interior point	true	C-2 happy path
T-3-10	`DataField3D::Contains`	point outside bbox	false	C-2 sad path
T-3-11	`DataField3D::Contains`	point = bbox boundary, eps = 1e-6	true with eps, false without	epsilon edge
T-3-12	`DataField3D::findAxisIndex_`	value = axis[0]	returns 0	left edge
T-3-13	`DataField3D::findAxisIndex_`	value = axis[Nx-1]	returns Nx-2	right edge

Phase 4 — `test_field_coefficient.cpp` + `test_field_projector.cpp`

ID	Function	Fixture	Expected	Guard
T-4-1	`FieldCoefficient::FieldCoefficient`	DataField3D for f=ax+by+cz+d; ParMesh covering interior of f's bbox	projection error → 0 with mesh refinement (linear exact at order 1)	trilinear ⊆ P1
T-4-2	`FieldCoefficient::scale_`	scale=2, offset=10	result = 2*raw + 10	scale/offset
T-4-3	`FieldProjector::FieldProjector`	sidecar bbox strictly contains mesh	success; min/max in bounds	C-2 happy path
T-4-4	`FieldProjector::FieldProjector`	sidecar bbox slightly smaller than mesh in x	abort with both bbox tables	C-2 sad path
T-4-5	`FieldProjector::FieldProjector`	sidecar Vp_max declared 9000 but mesh-projected max = 9100	abort with observed vs declared	post-project
T-4-6	`FieldProjector::FieldProjector`	Project $V_p = 3000$ m/s; $V_s = V_p/\sqrt{3}$	derived μ lands within `mu_min_pa`/`mu_max_pa` bounds	success
T-4-7	`FieldProjector::ProjectOnce`	call twice on a counting-style mock driver for 5 time steps	CallCount() == 1	C-6 (one-load)
T-4-8	parallel projection (np=4)	sidecar bbox contains mesh; identical seed	per-rank min/max identical to np=1	MPI

Phase 5 — `test_heterogeneous_material.cpp` + `test_tpv102_with_velocity_sidecar.cpp`

ID	Function	Fixture	Expected	Guard
T-5-1	`MaterialField::At` (Constant)	mode=Constant, λ=10, μ=20, ρ=30	returns (10, 20, 30)	scalar path
T-5-2	`MaterialField::At` (GridFunction)	mode=GridFunction with ρ_gf,λ_gf,μ_gf set to constants	returns the same constants per (elem, dof)	hetero path
T-5-3	`MaterialField::MaxCpInElement`	mesh with Vp varying 4000 → 6000 across element	returns max at 6000	per-element
T-5-4	`WaveOperator(constant ctor)`	TPV102 constant material	byte-identical to pre-change reference	regression
T-5-5	`WaveOperator(MaterialField ctor)`	MaterialField{Constant, λ, μ, ρ}	byte-identical to T-5-4	regression
T-5-6	`WaveOperator(MaterialField ctor)`	MaterialField{GridFunction} with constant ρ_gf etc.	output matches T-5-4 within 1e-12 per-DOF	hetero=const
T-5-7	`InitializeImpedancesFromMaterial`	constant MaterialField	DOFData impedances match the homogeneous TPV102 closed-form values	regression
T-5-8	`InitializeImpedancesFromMaterial`	heterogeneous MaterialField with two-layer step	upper-DOF impedance matches upper layer; lower-DOF matches lower layer	hetero
T-5-9	`tpv102_driver --velocity-sidecar` (smoke)	2000 m mesh + synthetic schema-v1 sidecar covering it	runs 100 steps without abort; CallCount()==1	E2E + C-6
T-5-10	`tpv102_driver --velocity-sidecar` smoke + bbox failure	sidecar bbox 10% smaller than mesh	aborts at startup with bbox table	C-2 E2E
T-5-11	`tpv102_driver --velocity-sidecar` smoke + low-Vs failure	sidecar with one Vs=100 cell	aborts at load with offending cell	C-3 E2E

Phase 1 — schema doc lint

ID	Subject	Check
T-1-1	`data_projection_schema_v1.md`	enumerates every attribute used by Phase 2 / Phase 3
T-1-2	`data_projection_schema_v1.md`	states “NaN forbidden”, “OOB policy = abort only”, “per-field min/max mandatory”
T-1-3	cross-cite	Phase 2 writer + Phase 3 reader doc strings cite the schema

---

Testing Strategy (summary)

Phase	Test type	What it covers
1	doc lint	schema doc covers every attribute used downstream
2	pytest unit + smoke	reader correctness, CRS round-trip, sidecar I/O, writer guards G-1 / G-2 / G-3
3	gtest unit	trilinear evaluation, schema gate, value-range gate, bbox query

Phase	Test type	What it covers
4	gtest unit + np=1/4	Coefficient eval, mesh-vs-data bbox guard, post-project bounds, one-load contract
5	gtest + TPV102 smoke	byte-identical when off; runs to completion when on; bbox-fail and low-Vs-fail E2E

Validation against analytics

For Phase 3, T-3-5 is the standard convergence-zero check: `f(x,y,z) = ax + by + cz + d` is exact under trilinear interpolation, verified at 1024 random points to 1e-12.

For Phase 5, T-5-6 is the homogeneous-equivalence check: heterogeneous `MaterialField` with constant GFs must produce output matching the scalar ctor to 1e-12 per-DOF.

Risk Assessment

High risk

- **Coordinate-frame slip** (Phase 2). A single cut-and-paste error in `geographic_to_utm11n` produces a silently shifted result of order 100 km. Mitigation: T-2-4 (round-trip) + T-2-5 (anchor calibration).
- **`Wave0perator` heterogeneity refactor breaking TPV104 / TPV205 / BP5.** Mitigation: T-5-4 / T-5-5 / T-5-6 form a three-way regression cordon.
- **Time-loop accidentally re-projects.** Mitigation: T-4-7 (call counter at unit level) + the runtime `MFEM_ASSERT(CallCount()==1)` inside the driver's per-step block.

Medium risk

- **HDF5 portability on Frontera.** The project's toolchain (ICX 2023.1 + IMPI 2021.9 + GCC 9.1, see `reference_seissol_frontera.md`) requires that HDF5 is linked consistently with MFEM. Check `mfem-config --libs` for `-lhdf5` before merging.
- **Mesh bbox `MPI_Allreduce` cost** at startup. Worst case 3 × MIN + 3 × MAX over all ranks; bounded.

Low risk

- **Velocity raster mask ambiguity.** Phase 2 §6 G-1 (NaN) and G-2 (per-field bounds) cover this twice over.
- **Schema version drift.** `schema_version = "data_projection_v1"` is checked at load (T-3-1). Future schema bumps add new tests.

Summary of New Files

```
miniapps/seas/document/features_dev/
  data_projection_feature_plan_v1.md      (preserved; superseded)
  data_projection_feature_plan_v2.md      (THIS FILE)
  data_projection_schema_v1.md            (Phase 1)

safs/project_7.0_alternative/code_preprocess/data_projection/
  __init__.py                             (Phase 2)
  raw_readers.py
  crs.py
  sidecar.py
  bbox_check.py                           (NEW in v2)
```

```
  build_velocity_cvmh.py
  test_data_projection.py

safs/project_7.0_alternative/data_projected/
  velocity_safs.h5                        (Phase 2 output, generated)

miniapps/seas/io/
  data_field_3d.hpp                       (Phase 3)
  data_field_3d.cpp                       (Phase 3, optional)
  field_coefficient.hpp                   (Phase 4)

miniapps/seas/dynamic/
  heterogeneous_material.hpp              (Phase 5)
  heterogeneous_material.cpp              (Phase 5)

miniapps/seas/test/
  test_data_field_3d.cpp                  (Phase 3)
  test_field_coefficient.cpp              (Phase 4)
  test_field_projector.cpp                (Phase 4)
  test_heterogeneous_material.cpp         (Phase 5)
  test_tpv102_with_velocity_sidecar.cpp   (Phase 5)
```

Files Modified (additive only)

```
miniapps/seas/CMakeLists.txt                    (Phase 3–5: add sources & tests)
miniapps/seas/Makefile                           (Phase 3–5)
miniapps/seas/dynamic/wave_operator.hpp         (Phase 5: new ctor)
miniapps/seas/dynamic/wave_operator.inl         (Phase 5: At() dispatch)
miniapps/seas/dynamic/fault_face_flux.cpp       (Phase 5: impedance helper)
miniapps/seas/drivers/tpv102_driver.cpp         (Phase 5: CLI flag)
```

No edits to: `bp5/`, `bp1/`, `bp2/`, `domain/`, `friction/dieterich_ruina.hpp`, `safs/CFM_data_step/`, any `.toml` example, or any other driver. C2 invariant preserved.